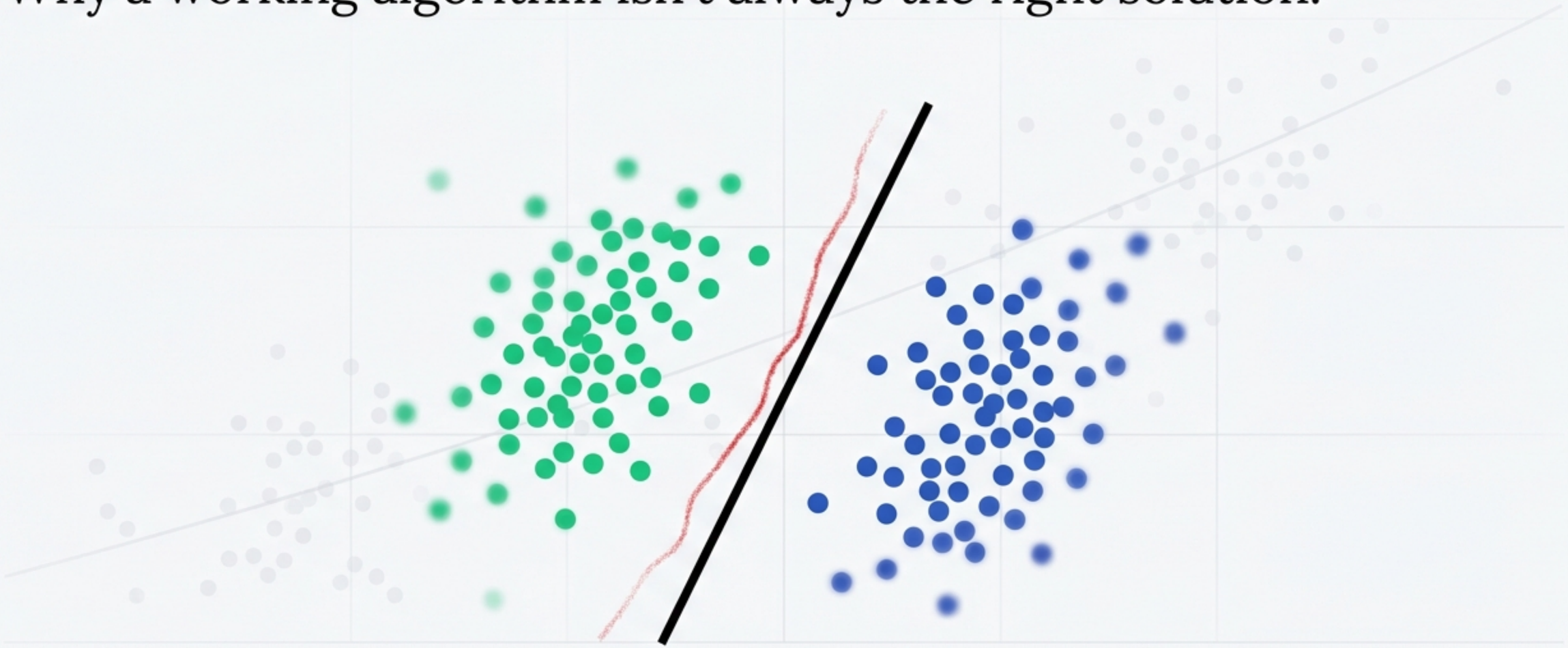
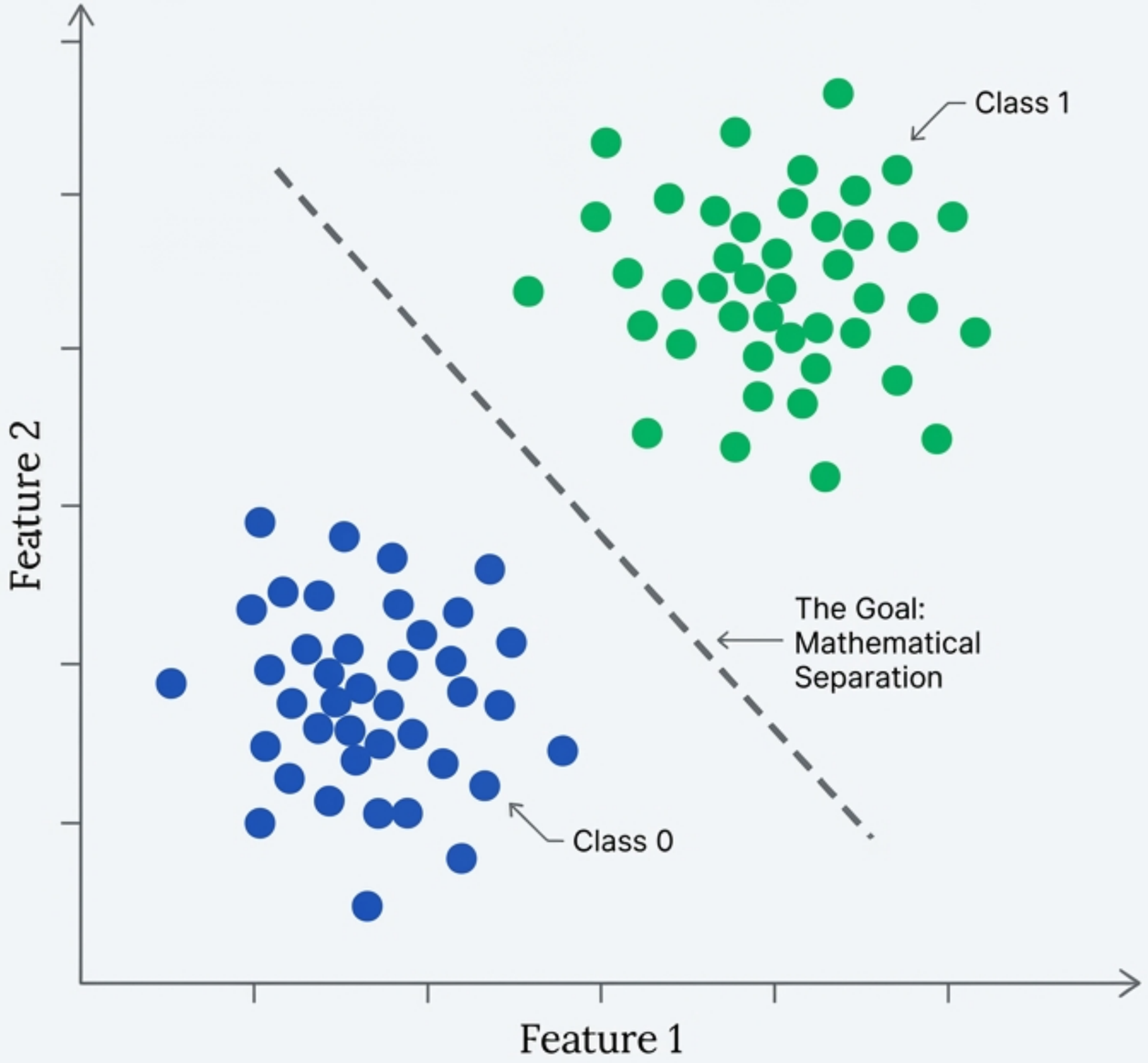
Coding the Perceptron Trick

Why a working algorithm isn’t always the right solution.



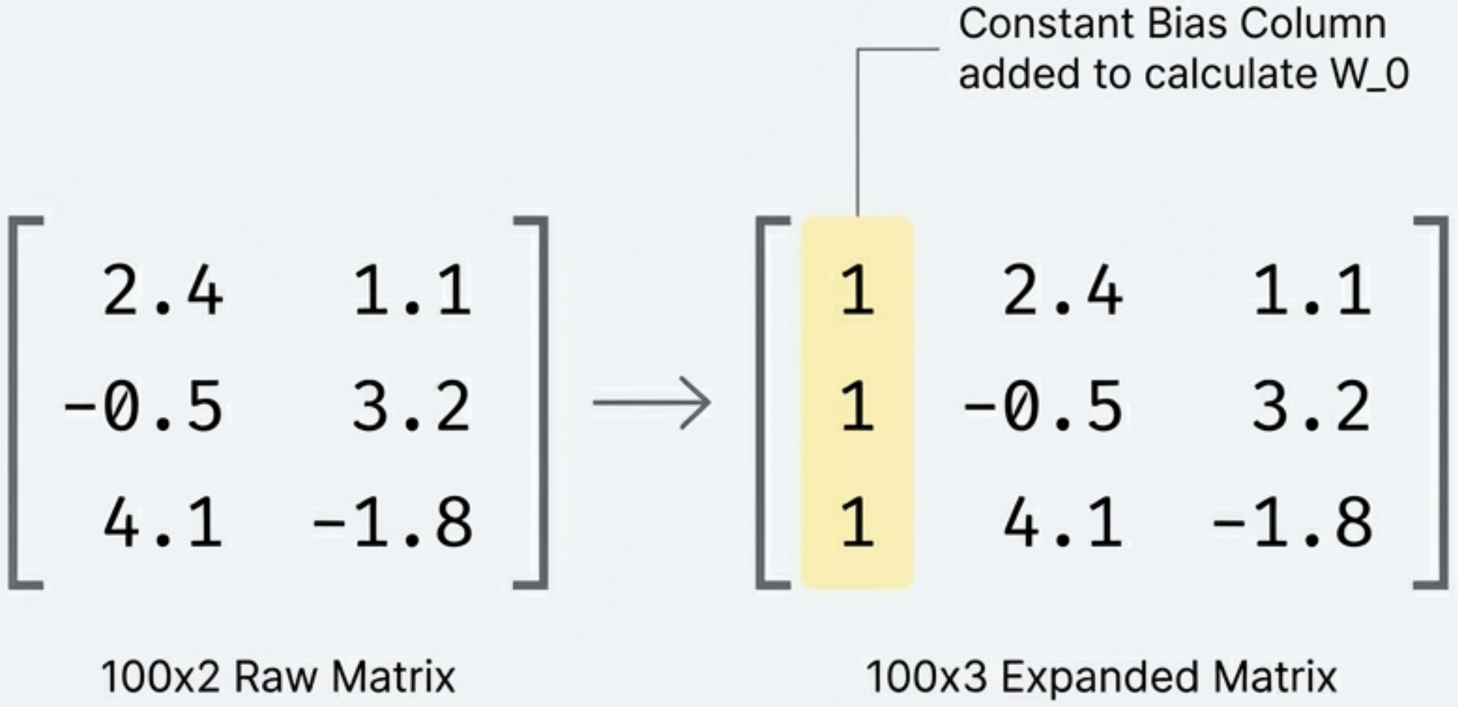
The Objective: Separating 2D Data

We begin with a randomly generated classification dataset comprising 100 data points across two features. The target variable is binary.



Step 1: Implementing the Bias Trick

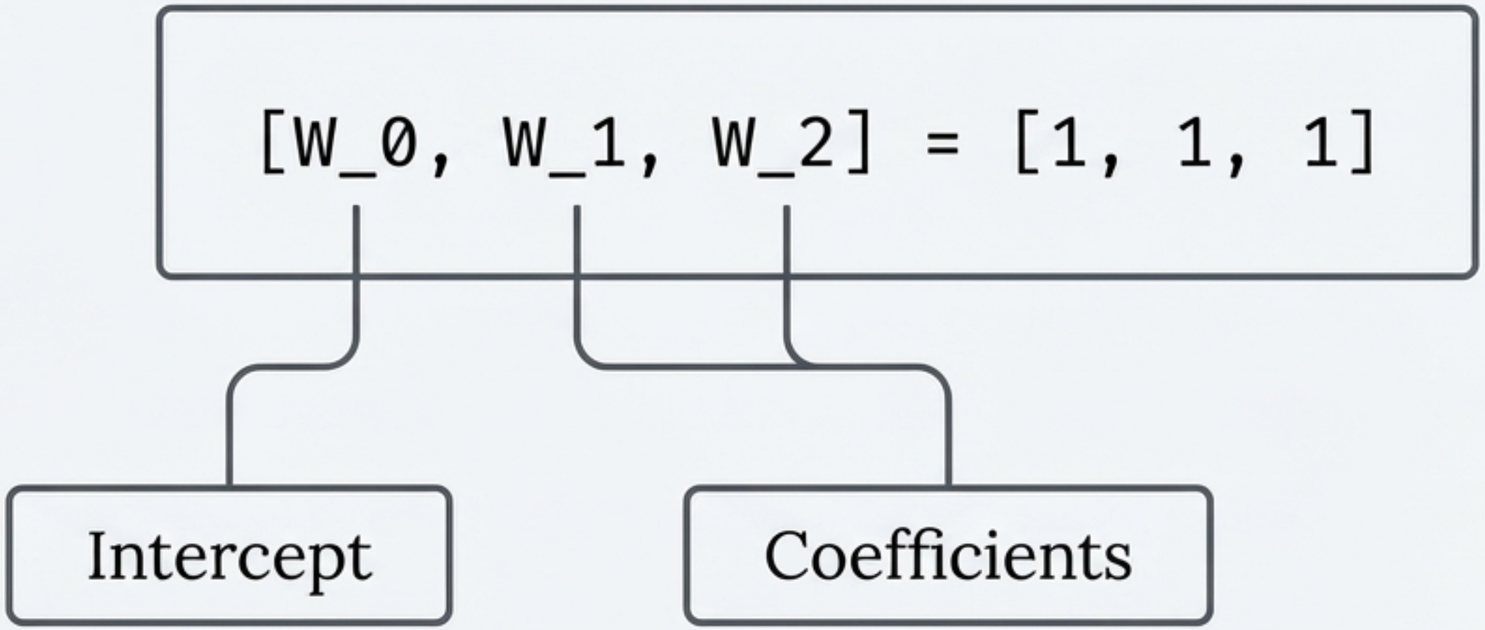
To handle the Y-intercept seamlessly within our matrix math, we append a constant column of 1s to the input data.



Step 2: Initialising the Weights

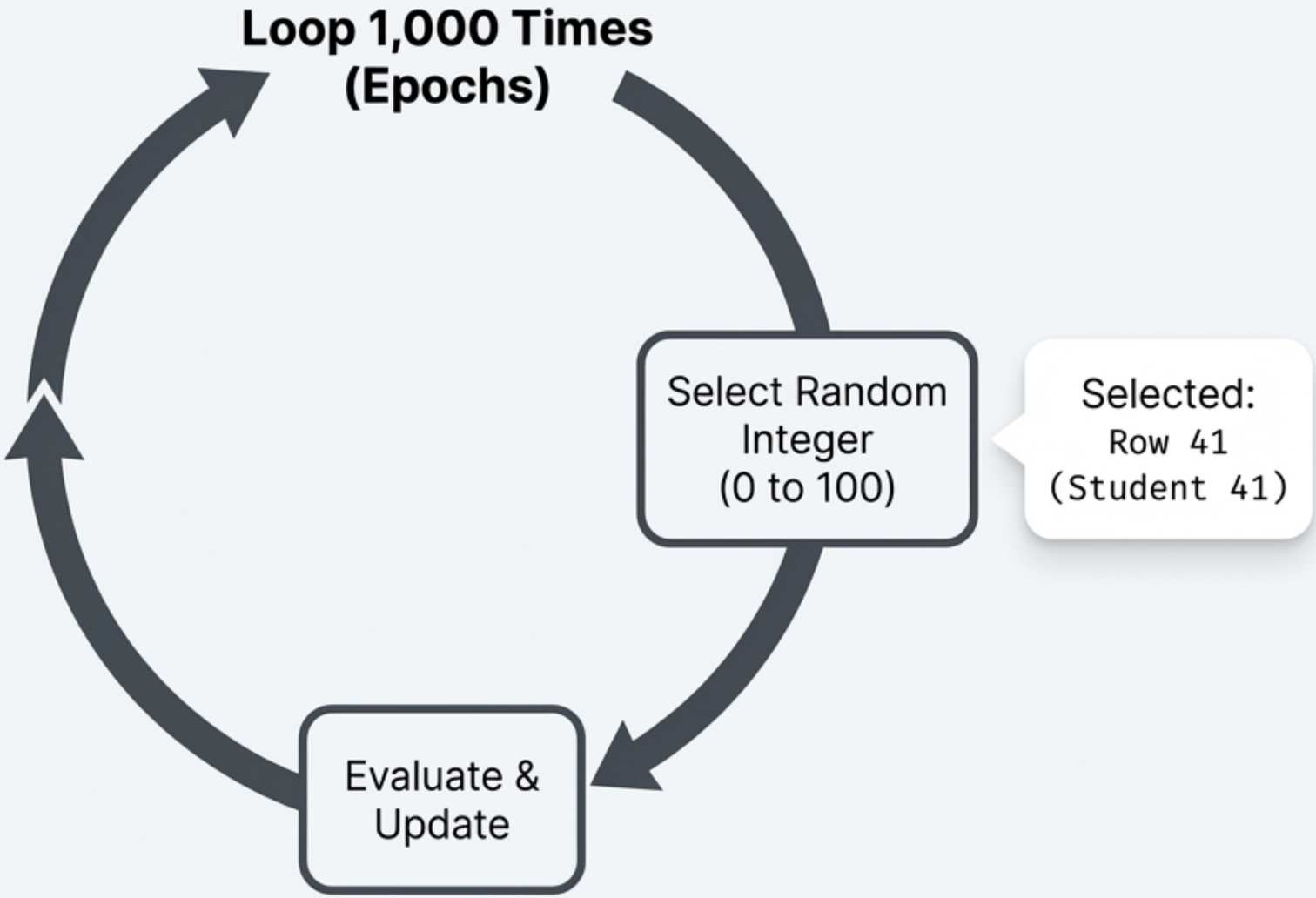
We generate a weight array to match our newly expanded matrix. The learning rate dictates how aggressively the line moves when it makes a mistake.

Learning Rate = 0.1



Step 3: The Stochastic Training Loop

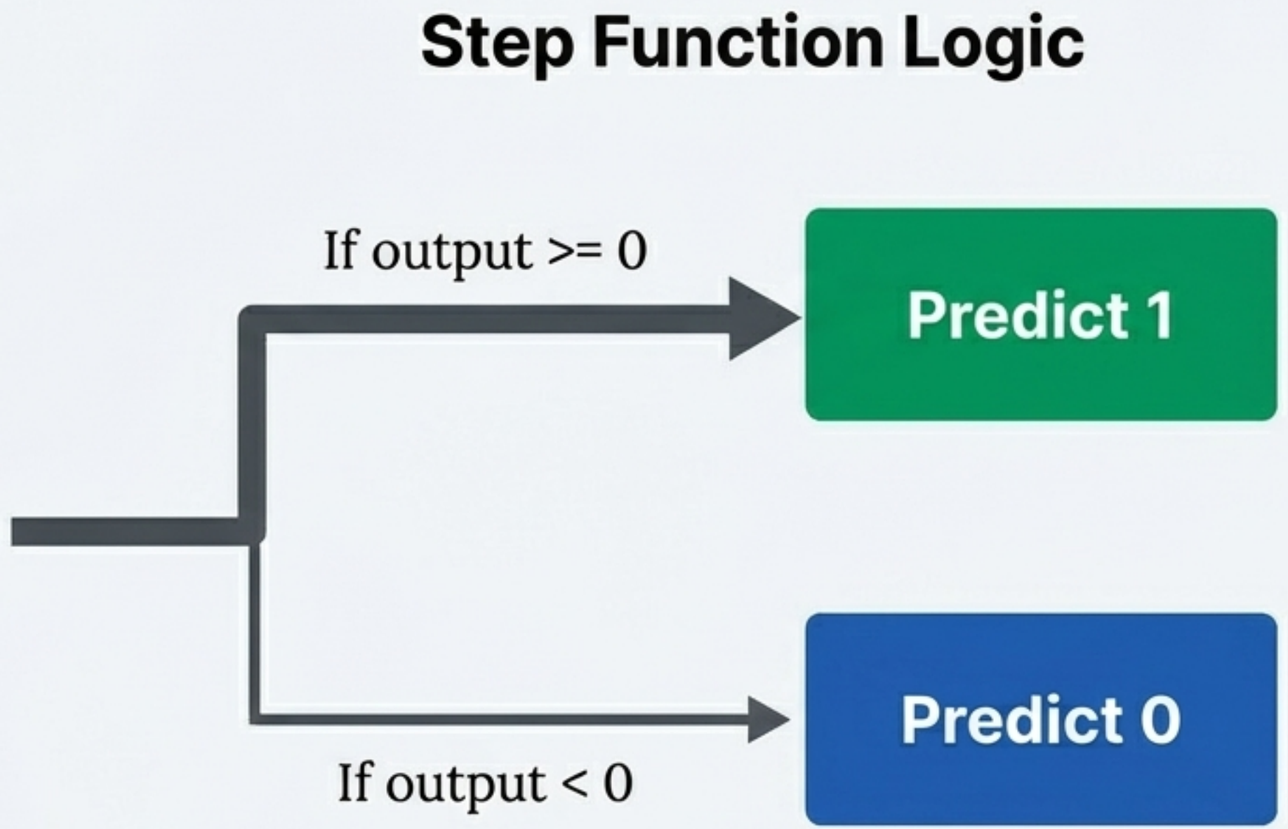
Instead of evaluating the entire dataset at once, the algorithm trains for 1,000 epochs. In each loop, it randomly selects **exactly one** data point.



Step 4: Making a Prediction

The model calculates the dot product of the weights and the selected data point. A simple Step Function dictates the predicted class.

`Weights · Row 41 = 0.29`



Step 5: The Weight Update Logic

The weights only change when the prediction is wrong. If the model guesses correctly, the multiplier becomes zero, and the boundary remains static.

$$\text{New Weights} = \text{Old Weights} + 0.1 * (\text{Actual} - \text{Predicted}) * X$$

If Actual is 1 and Predicted is 1, this term equals 0. No update occurs.

Bridging Mathematics: Arrays to Boundaries

To visualise our final weights (A, B, C) as a decision boundary on our scatter plot, we convert them into the standard line equation format: y = mx + c.

Model Outputs

`Weights Array: [A, B, C]`

A: First Coefficient

B: Second Coefficient

C: Intercept

Line Equation Form

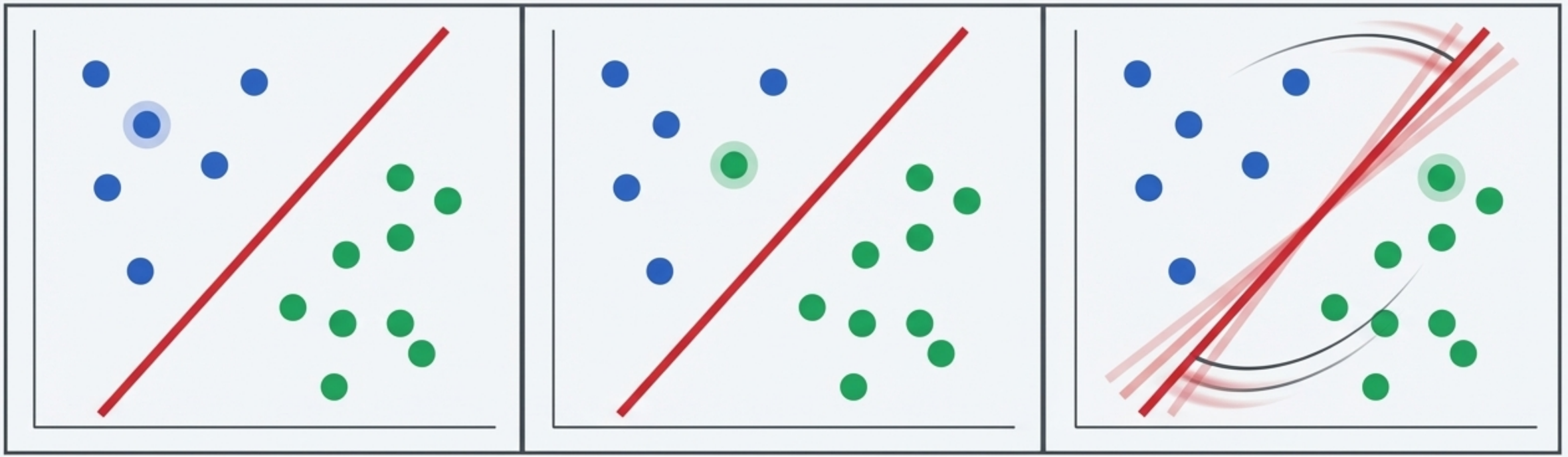
`y = mx + c`

`Slope (m) = -A / B`

`Intercept (c) = -C / B`

Stop-Motion Learning in Action

Observing the training animation reveals the Perceptron's reactive nature.



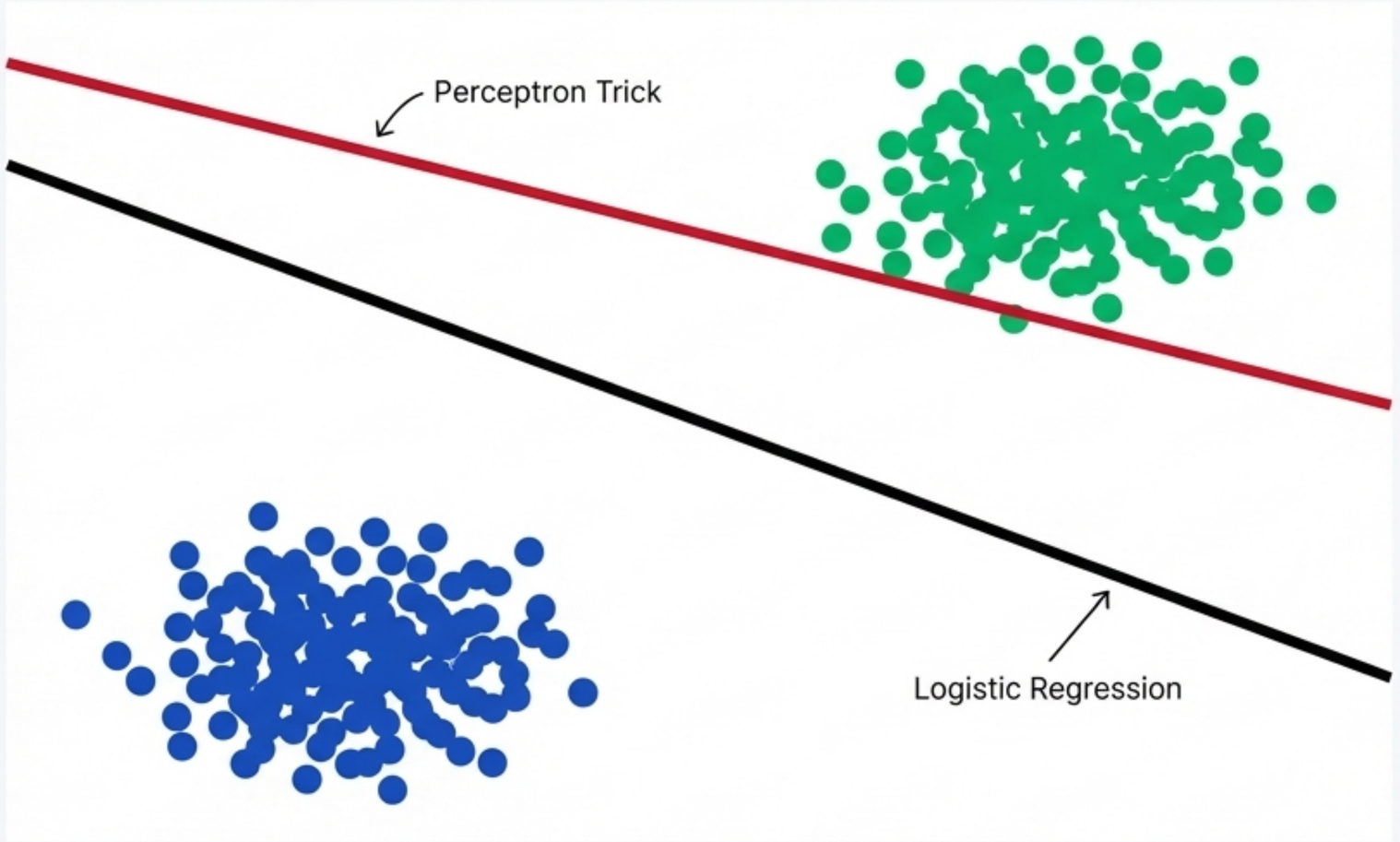
1. Correct Prediction: Line remains static.

2. Encounters Error: Misclassified point.

3. Reactive Update: Line jerks to fix error.

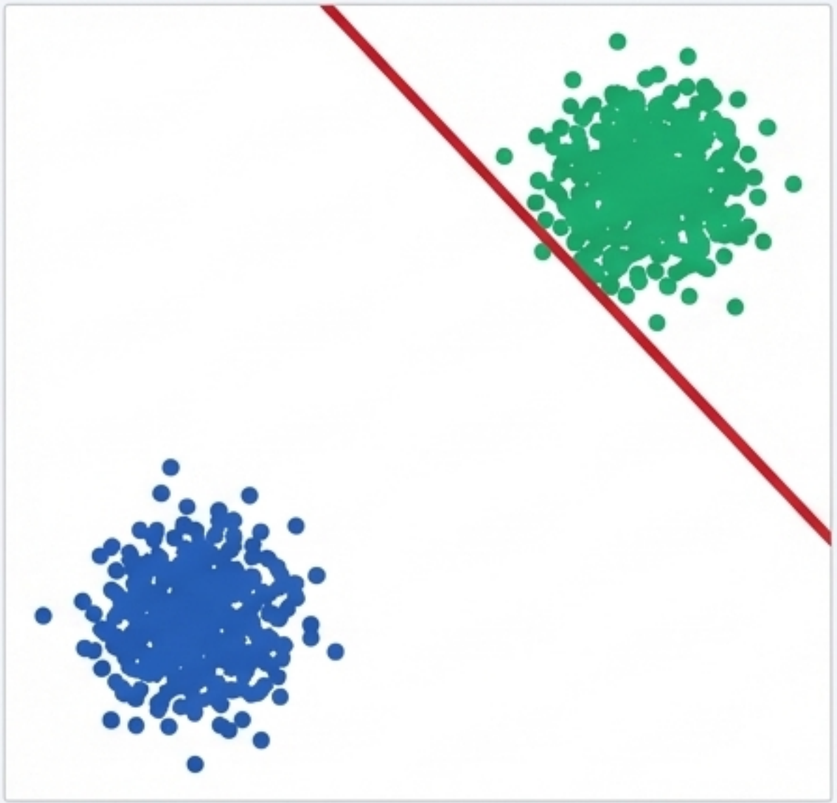
The Visual Climax: Two Lines Diverge

When comparing our custom Perceptron trick to Scikit-Learn's official Logistic Regression implementation, a clear discrepancy emerges.

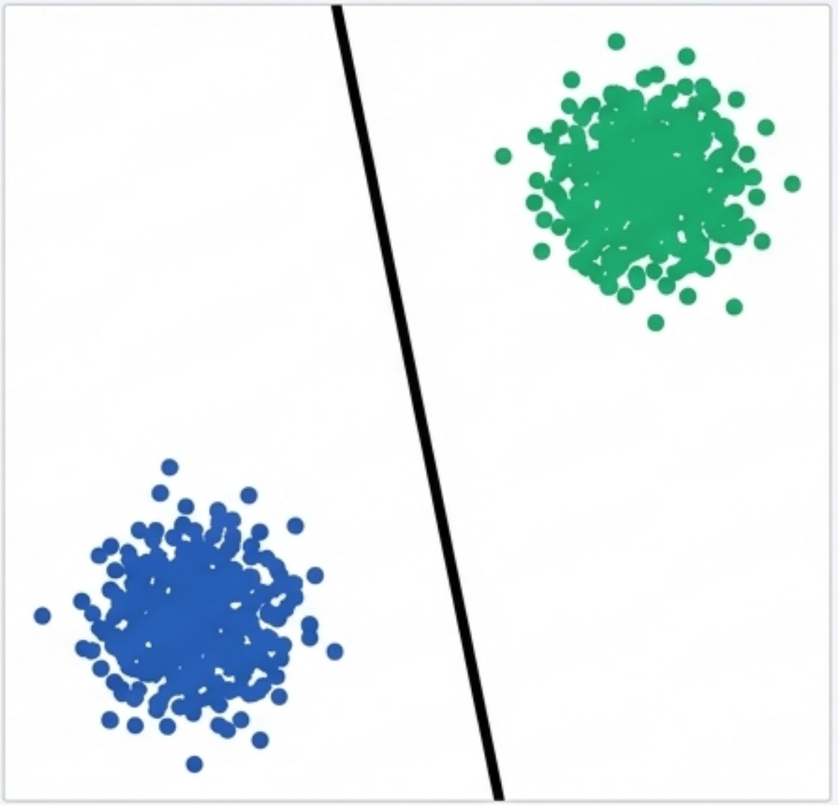


Stress-Testing the Margins

Increasing the separation distance between the two classes exposes the critical flaw in the Perceptron algorithm.



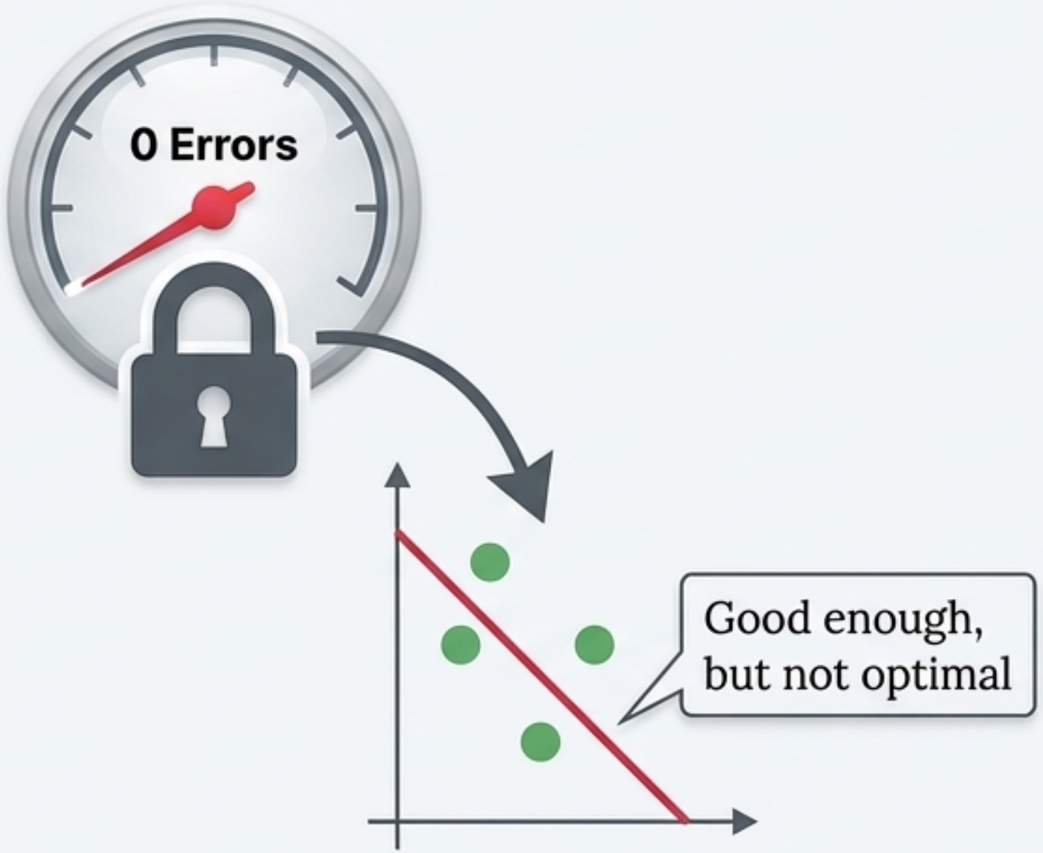
Perceptron Boundary



Logistic Regression Boundary

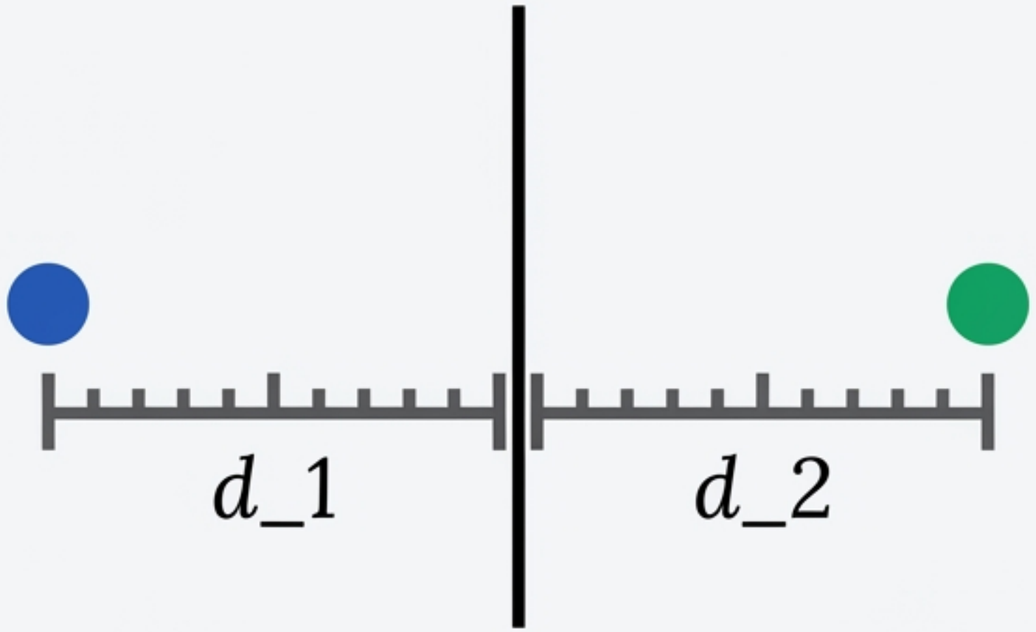
The Problem with a ‘Lazy’ Learner

The Perceptron rule halts the exact moment misclassifications hit zero. Once every point is on the “correct” side, it stops learning. It never seeks to improve the boundary.



Symmetry, Margins, and Overfitting

True Logistic Regression pushes for the optimal, symmetrical margin between classes. Equal distance prevents overfitting and ensures much lower error rates when tested on unseen data.



$$d\_1 = d\_2$$

✅ Better Generalisation

The Evolution of an Algorithm

The Perceptron trick is a foundational, elegant piece of code for understanding linear boundaries. However, data science relies on **Logistic Regressic** Regression to symmantete guarantee symmetrical, optimal margins that survive contact with the real world.

```
np.dot(weights, features) = np.dot(weights,features)
for i in range(epochs):
    error = target - prediction
    error = target - prediction
    weights += learning_rate * error * features

[[X1, X2],  [w1]
 [X3, X4]]  [w2]

np.dot (weights, features)
for i in range(epochs):
    error = target - prediction
```